

Stochastic techniques for resource management

C Brind, C Muller and P Prosser

This paper investigates a number of stochastic search techniques applied to vehicle routing problems (VRPs) with time-windows and technological constraints. Five techniques are investigated — genetic algorithms (GA), hill-climbing (HC), random search (RS), simulated annealing (SA), and tabu search (TS). Their performance is examined and compared over a wide range of VRPs with varying degrees of workforce specialisation, job time-windows and personnel resources. The objective of the study is twofold — firstly, to examine the behaviour of the techniques and to identify the best, and secondly to examine features of the problem in such a way that the behaviour of the techniques and possibly the quality of the solutions can be anticipated.

1. Introduction to the vehicle routing problem

A vehicle routing problem (VRP) consists of a workforce operating from one base and a set of geographically distributed customers requesting a service. The objective is to maximise the amount of work done and minimise the total distance travelled. In this study, the original problem has been further complicated by the introduction of two side-constraints:

- some jobs can only be done during some well-defined time-windows,
- some jobs can only be performed by a limited subset of the workforce.

Figures 1a and 1b give a graphical representation of a VRP and one possible solution. The grid (in the middle of Figs 1a and 1b) represents the base (where the engineers start and finish their tour) and the diamonds represent the 200 jobs. All these elements are contained within a 5 km radius of a central location. Figure 1b shows the tours of the different engineers. It should be observed that, in this case, all the jobs have been allocated. This problem is 50-0-0-3 (see next section) and is over- resourced, with no (technological or temporal) constraints. In the case of an under- resourced problem (with or without constraints), some jobs would have remained unallocated.

The vehicle routing problem can be considered as a more general instance of the travelling salesman's problem (TSP). In the TSP a travelling salesman wishes to visit a number of cities and return to a starting point, in such a way that each city is visited exactly once, and the total distance covered is as short as possible (the least distance Hamiltonian tour). In the VRP there are many salesmen,

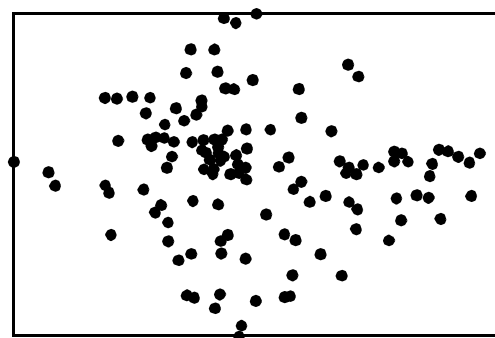


Fig 1a A VRP problem.n

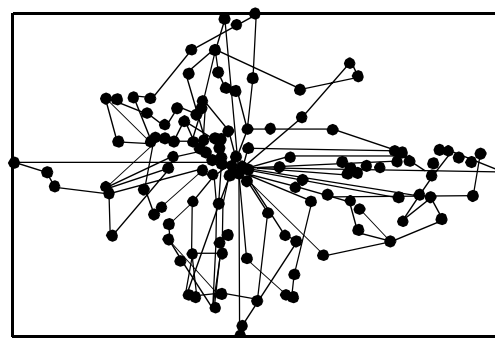


Fig 1b A possible solution.

consequently the TSP is a VRP with a single salesman. Furthermore, in the VRP presented here there are both technological and temporal constraints.

1.1 Workforce management as a VRP

The workforce management problem is modelled as a VRP as follows. There is a set of jobs, where each job has a

location, a duration, a time window in which the job must be performed, and a set of engineers that can perform that job. There is also a set of engineers and a base location.

Jobs were created with locations drawn randomly (using a uniform distribution) from a file of postal codes with corresponding x/y co-ordinates (measured in decametres). A file of a city's postal codes in a 5 kilometre radius of the city centre was used (referred to as the G5K data set), giving an urban VRP. The durations of jobs (in minutes) was drawn randomly (again using a uniform distribution) from the set {15, 30, 45, 60, 75, 90, 105, 120}. A time-window was assigned to each job constraining when that job could be performed. Jobs could be created with temporal constraints (time windows) drawn from the following distributions:

t0 all jobs have a time window running from 9 to 5 (all day) and corresponds to no temporal constraint,

t1 probability of morning (9 to 12) is 0.4, probability of afternoon (12 till 5) is 0.4, and probability of all day (9 to 5) is 0.2,

t2 probability of morning is 0.45, probability of afternoon is 0.45, and probability of all day is 0.1.

Similarly, a technological constraint was assigned to each job constraining which engineers could do that job. Jobs could be created with technological constraints (in the form of engineer specialisations) drawn from the following distributions:

s0 all engineers can do any job (and this corresponds to no technological constraints),

s 80% of jobs can be done by 80% of the engineers, and 20% can be performed by 20% of the engineers (and this corresponds to problems with relatively loose technological constraints),

s2 50% of the jobs can be done by 80% of the engineers, and 50% of the jobs can be done by 20% of the engineers (a moderate technological constraint),

s3 80% of the jobs can be done by 20% of the engineers, and 20% of the jobs can be performed by 80% of the engineers (and this corresponds to problems with tight technological constraints).

Therefore, a VRP can have a number of engineers, a number of jobs, and temporal and technological constraints. All problems generated had 200 jobs drawn from G5K, with durations as above, and it was assumed that engineers travel at 12 mph across Manhattan distances. The number of engineers was allowed to vary, as was the temporal and technological constraints. Therefore, a VRP is given an identity (e- t-s-n) where:

e is the number of engineers, typically 30, 40, or 50,

t is the time window distribution, 0 for t0, 1 for t1, and so on,

s is the specialisation distribution, 0 for s0, 1 for s1, and so on,

n is a sequence number.

For example problem 40-0-0-2 is a VRP with 40 engineers, 200 jobs (default) in the G5K region (again default), no time windows (every job can be done between 9 and 5), no specialisation (every engineer can do any job), and it is the second problem generated with these characteristics¹. This, therefore, gives under-resourced problems (i.e. 30-*-**) and over-resourced problems (50-*-**), loose (*-0-**) and tight temporal constraints (*- 2-*-*), and loose (*-*0-*) and tight technological constraints (*-*3-*). Initially problems were generated to cover all combinations of parameters, from underresourced and unconstrained through to over-resourced and tightly constrained, and at each parameter setting three problems were generated. In total this gives a database of 108 problems. Of these 108 problems 36 were used in the experiments, and the following problems were ignored: *-1-**, *-*-1-*, *-*-2-*, i.e. only problems with time windows t0 or t2 were used, and problems with specialisation s0 or s3. This gives a set of extreme problems, that are combinations of under-resourced to over-resourced, and relaxed to restrictive temporal constraints, and relaxed to restrictive technological constraints.

1.2 An active constraint-based model

All the search techniques described below use a common representation of the VRP. A solution to a VRP is represented as a list of engineers, and for each engineer an ordered list of jobs (a tour). For an n engineer problem there is always an $n+1$ th engineer (the virtual engineer) that is able to do all work. Therefore, if a job has not yet been allocated to any of the n engineers it will reside in the tour of the $n+1$ th engineer. This tour is frequently referred to as the 'conflict list' or as the set of 'jobs not done'. One of the goals is then to minimise the duration of 'jobs not done' (described in more detail later).

A constraint-based approach is taken when representing an engineer's tour. A tour is represented as a sequence of jobs, such that it is possible to perform each job within its time window, travel between jobs in the given sequence, and return to the base before the end of day (i.e. a legal tour using a least commitment strategy). When a job is inserted into a tour constraint, propagation takes place, updating

¹ The problem 1-0-0-1 would be a 200 city TSP, assuming job durations are ignored.

(restricting) the time windows of jobs within that tour. Symmetrically, when a job is removed from a tour, constraints are retracted, and time windows may be relaxed. Therefore, central to the representation of the VRP is a constraint maintenance system, giving an active representation of tours.

1.3 The cost of a solution

The principle objective is to perform all the jobs and a second objective is to minimise the total distance travelled by the workforce. The following cost function is used to attribute a cost for each solution:

$$\text{cost}(S) = C \cdot D(S) + T(S)$$

where C is a constant,

$D(S)$ is the sum of the durations of the jobs not done (in minutes),

$T(S)$ is the sum of the travel times (in minutes) of the engineers.

The travel time for an engineer assumes Manhattan distance covered at a given speed (in this case 12 mph), and is measured in minutes. Given the two locations $\langle x_1, y_1 \rangle$ and $\langle x_2, y_2 \rangle$ the Manhattan distance is $|x_1 - x_2| + |y_1 - y_2|$.

In some early tests, a small value had been chosen for C ($C=3$). However, in some instances, the different algorithms were trading work for travel, i.e. algorithms could get a better cost by doing less work (and hence, much less travel). This situation was undesirable. The objective of performing all the jobs is indeed more important than the reduction of travel, and the cost function has to reflect this obligation. Consequently, a large value was chosen for C .

By choosing a large enough value for the coefficient C it is hoped that it is not possible to trade travel for work done. Put another way, given any two solutions S' and S'' , where $\text{cost}(S') < \text{cost}(S'')$, $D(S') \leq D(S'')$. Therefore, an improvement can be made to S by increasing the amount of work done (reduce the amount of work not done), or by reducing the amount of travel. It should be impossible to reduce the cost of S by reducing the amount of work done in S (increasing the amount of work not done in S).

All the experiments were run using a coefficient $C=600$. Hence, the cost of a solution is 600 times the durations of the jobs not done plus the different travels. Unfortunately this gives a cost function that goes through an abrupt change when all jobs get done. If there are two solutions and one of these manages to allocate more jobs than another, then difference in cost between these two solutions will be enormous (i.e. at least 600×15). However, when comparing two solutions to over- resourced problems (i.e.

$D(S)=0$), differences in costs should be small (i.e. only due to differences in travel). Back to contents

2. Techniques

Reports are available which discuss the techniques mentioned here more fully [1, 2], and also the particular instances of the techniques used for this paper [3, 4].

2.1 Random search (RS)

RS is used as a reference technique, against which the improvements gained by using the other techniques could be usefully measured. It also demonstrates certain features of the problem, such as the difficulty of finding a solution where all work is done. RS operates by performing a random sampling of the search space and keeps a record of the best solution found during the search. It enters a loop and at each iteration the algorithm creates a random solution via the schedule builder, and then evaluates that solution. The cost of the current solution S is then compared with the cost of the best solution found so far (S_{best}). If $\text{cost}(S) < \text{cost}(S_{best})$ then S_{best} is replaced by S , otherwise S_{best} remains unchanged and S is relinquished. This cycle is repeated until the time limit is reached.

RS uses a schedule builder to generate random solutions. The initial schedule builder was given a randomly ordered list of engineers, it then took each job in turn, attempting to allocate the job to each engineer in the order in the list, only moving on to the next if the current engineer could not accommodate the job. This gave poor diversity of solutions; in the extreme case, where there is no workforce specialisation, on different executions it produced the same set of tours, but allocated them to different engineers. This lack of diversity penalised RS, and the schedule builder was also used by the genetic algorithm (GA) to produce the initial population (and all of the indirect GA's solutions). This led to the design of a new schedule builder which also randomised the order of allocation of jobs, and so produced more diverse solutions. This second schedule builder was used for all of the experiments reported here.

It should be noticed that the schedule-builder works with an engineer-centred perspective. This strategy is similar to the resource-centred perspective adopted by OPIS [5]. A job-centred perspective could also have been adopted. A schedule-builder follows an ordered list of jobs and attempts to attach each job to an engineer. This approach is similar to the operation-centred perspective used in ISIS [5].

A job-centred schedule-builder was implemented in a previous project [7]; the engineer-centred algorithm consistently produced solutions of better quality. Hence, it was decided that a job-centred schedule-builder was not desirable for the present study.

2.2 Hill climbing (HC)

Hill climbing starts with an initial, random solution and tries to alter it, accepting only those alterations which improve the solution's cost. HC generates $n + 1$ neighbours (at each iteration), where n is the number of engineers, around the current solution S . Once this has been completed, the best solution S_{best} in the neighbourhood is compared with S . The difference of cost between S and S_{best} is calculated. If S_{best} has a lower cost, the move is accepted and S_{best} becomes S , otherwise S remains unchanged. This cycle is then repeated until the time limit is reached.

2.3 Simulated annealing (SA)

Simulated annealing was introduced by Kirkpatrick [8] and is analogous to the physical process of annealing, where a solid is heated to a high temperature and then slowly cooled to produce a regular crystal structure. To apply this to state-space search, temperature is linked with the disruptiveness of alteration allowed, and a crystal structure corresponds to a low-cost solution.

The SA only samples one neighbour from the neighbourhood at each step. SA always accepts moves which improve the cost of the current solution and only accepts moves which do not improve the cost with a probability of:

$$e^{-\frac{\Delta}{T}}$$

where Δ is the difference in the cost of the solution caused by the move and T is the current temperature.

It is common to allow SA to reach equilibrium before reducing temperature and this is often measured as completing a specified number of iterations. SA also requires an annealing schedule which is comprised of the initial temperature and the cooling rate. SA usually starts at a relatively high initial temperature which allows highly disruptive moves. Rather than reducing the temperature by a fixed amount at each stage, it cools by multiplying the temperature by the cooling rate; this technique gives fast initial cooling but a slower cooling as the search progresses. As the temperature approaches 0 disruptive moves are unlikely to be accepted and the search's behaviour approximates to that of a hill climber, only accepting improving moves.

A set of calibrating experiments were performed and the parameters which appeared to perform well were:

- equilibrium was defined as completing 500 successful moves,
- the initial temperature used was 10,
- the cooling rate used was to multiply the temperature by 0.992 at each reduction.

2.4 Tabu search (TS)

Tabu search was created by Glover [9, 10] and is a neighbourhood technique. Its analogy is with memory - it remembers where it has searched recently and does not revisit these solutions, as well as temporarily holding certain aspects of the current solution constant. This will prevent cycling, as well as adding vigour to the search.

When TS accepts a move to a new solution, it stores attributes of that move in the tabu list. The attributes are removed from the tabu list after a predefined number of moves; this number is known as the tabu tenure. A move is said to be tabu if there is an attribute in the tabu list which fulfills the tabu criteria for that move. TS samples several solutions from the neighbourhood at each step and then chooses the non-tabu solution with the best cost.

Usually more general criteria are used than simply preventing a return to single solutions. TS will usually disallow a 'class' of solutions with a particular characteristic, e.g. disallow any solution which does not have a particular job in a particular place. This temporarily enables a great reduction of the search space, and often relates well to attributes of the moves performed.

The parameters for TS are the tenure (tabu list size), attributes stored and the tabu criterion. All moves made have attributes which describe certain details of the move, such as the job and the engineers involved, and these attributes are stored in the tabu list. The tabu list has a fixed size so when a new move is recorded the oldest set of attributes are lost. Only a reasonable amount of the search-space should be restricted, the more general the tabu criterion the shorter the tenure.

The calibrating experiments suggested the following parameters:

- the attribute used was the identifier of the job moved,
- the tabu criterion used was not to move the job again within tenure,
- and the tabu tenure used was 125.

The neighbourhood move was defined as follows. For each engineer the move used was to select a job, if there was one, from their tour and move it to a new position in the solution. As with all the neighbourhood techniques, TS uses a double move for the conflict list (see above).

2.5 Genetic algorithms (GA)

Genetic algorithms (GA) are modelled on Darwinian evolution, the survival of the fittest [11-14]. There are many flavours of GAs, most typically the generational model and the steady-state model.

Under the generational model, initially there is a population of chromosomes, where each chromosome represents a solution of the problem under study. Chromosomes are then ranked according to fitness (where the fitness of a chromosome is directly related to the quality of the solution it represents) and then mating takes place between chromosomes to produce a new population (a generation), where the allocation of mates is in proportion to the chromosomes' individual fitness. Consequently, the features that make up the fittest chromosome will gradually propagate through the entire population, and this is referred to as reproduction with emphasis (an exploitable mode of search, sometimes called convergent search) and the population as a whole becomes similar (homogenised) with a consequent loss of diversity. Diversity can be maintained, up to a point, by injecting a random element into the process, namely mutation - when mating takes place via the primary genetic operation (crossover) a new chromosome is created, and some random disturbance of that chromosome can be induced with low priority in order to maintain diversity. This corresponds to exploratory search (sometimes called divergent search).

The GA reported in this study used a steady-state model. Under this model, a GA works as follows. Initially, a set of n solutions is generated and sorted by fitness. At this point, a number in the range 1 to $n-1$ is generated randomly, call it x . Then, a number y is generated randomly where $1 < y \leq x$ and the y th chromosome (of the population) is selected for reproduction. Observe that this selection technique gives an exponential chance to the fittest elements. Once two parents have been selected (using the previous technique), the crossover operator generates one child which is inserted (at the correct position) in the population and the weakest member of the population is then deleted. This cycle of selection, crossover and replacement is repeated until the time limit is reached.

Traditionally (when applied to resource allocation problems), a GA works on an indirect representation of the problem and is used in association with a schedule builder. In this case, an indirect representation of a VRP could be a sequence of engineers. Alternatively, a direct representation can be employed where the chromosome and the solution to the VRP are the same thing, and this is the approach that has been adopted here. (Note that this was not an arbitrary decision. A comparative study was performed and it was shown that the indirect representation performed little better than random search, as previously reported [4].) In GA terminology, each element (or locus) of a chromosome represents one engineer and each one of these locus can be assigned different tours (or alleles). During a crossover operation, parts of two parents are mixed together to produce a child. On average, each child inherits half its tours from its parents (due to omissions and duplications during the inheritance process); a repair technique is used to reconstruct the second half of the child. This crossover

operator is more fully explained elsewhere [4]. With this direct approach, a chromosome can inherit random parts of its parents but it can also inherit the best parts, i.e. the crossover operator identifies the best parts (i.e. tours) of a schedule and favours their propagation (or inheritance).

The GA reported in this study uses a steady-state model, an exponential selection function, a direct chromosomal representation, a knowledge-based crossover (which tries to preserve tours with large amount of work being done) and works with a pool of either 400 or 2000 chromosomes (400 for 10 minute beauty contest and 2000 for CPU intensive experiments).

2.6 The neighbourhood

The neighbourhood techniques (namely HC, SA and TS), make a small alteration to the current solution S , to produce a new solution S' ; the complete set of possible values of S' is the neighbourhood. Initially the alteration which defined the neighbourhood was to remove a job from the solution and randomly allocate it to one of the engineers (including the virtual engineer), placing it in the earliest valid position in the engineer's tour.

If there are too few engineers to perform all the jobs then the solution quickly becomes packed with little space to move jobs about and so the virtual engineer must be used to take jobs out of the solution, enabling a more radical rearrangement of jobs, otherwise the search will lose vigour. The cost function used puts a great penalty on work left undone to the extent that a move which lowers the amount of work done is almost certainly doomed to be rejected. This leaves several useless solutions in the neighbourhood which the searches will waste time exploring and rejecting as well as preventing the removal of jobs from the solution to enable the allocation of others. To combat this, if a job is moved from engineer E to the virtual engineer then one of the virtual engineer's jobs is immediately allocated back to engineer E , thus ensuring that the total number of jobs done does not fall. The extra move out of the conflict list enables swaps where an equal or greater amount of work is done, though a swap which reduces the amount of work done will still fail. All of the neighbourhood techniques use this compound move and showed a significant improvement over the original strategy.

3. The 10 minute beauty contest

This section examines and compares the performances of GA, HC, RS, SA and TS. The computational resource was limited to 10 minutes of CPU time on a SPARCstation IPC per run. The initial temperature for SA was set to $Temp=10.0$, the cooling rate was $R=0.992$ and the number of successful moves at a given temperature was 500. The tabu

tenure for TS was set to 125 moves and the aspiration criteria was that the cost of a new solution had to be less than the best cost found so far.

The GA candidate used a knowledge-based crossover (attempting to preserve tours with a large amount of work done) and a steady-state model; it had no mutation operator and operated with a population of 400 chromosomes.

GA accepted duplicates but only during the initial building of the population. Thereafter, duplicates were rejected.

Table 1 gives the average cost reached by the five algorithms over the 30-*-* problems. Typically, the 30-*-* (or 30-* for short) problems are under-resourced, and all the algorithms generate solutions with large costs. The best solution is simply the one which minimises the amount of work not done (travel is only a secondary objective here). The 50-* problems are over-resourced, all the jobs are done and all the algorithms generate solutions with small cost (around 1000 units). Here, the reduction of travel is the primary concern and the best solution is the one that minimises travel (since all the jobs are allocated, travel is the only scope for improvement). For the 40-* problems, the situation is more delicate. Some of these problems have solutions in which all the work is done, and some have solutions in which some jobs remain unallocated. Clearly, these problems are critically resourced. Table 2 gives the ranking of the five algorithms using the following algorithm to rank them. Given a problem instance P_i and the five algorithms, each algorithm A_i is compared against the others. If an algorithm A_i finds a better cost than an algorithm A_j then the algorithm A_i is awarded one point. If the two algorithms find solutions of equal value, then they both get zero points. In total, an algorithm can score a maximum of four points and a minimum of 0 points for a given problem instance.

SA (with an impressive score of 3.88) dominates its opponents. SA is, in fact, only beaten twice (by GA and TS) on the 30-0-3-1 problem and the 30-2-3-3 problem. GA performs relatively well on under-resourced problems (in most cases, GA is ahead of HC, RS and TS). However, GA achieves some rather poor results when applied to over-resourced problems where, typically, GA is only able to beat RS. This explains the low score (1.58) of GA. GA is quite capable of packing jobs into under-resourced schedules; however, it cannot reduce travel as well as HC, SA, and TS.

The poor performances of GA on over-resourced problems are due to a number of causes. Firstly, the knowledge-based direct crossover used for this study is

Table 1 Beauty contest — average final cost.

	GA	HC	RS	SA	TS
30-0-0-1	535779	580803	975696	440246	559202
30-0-0-2	308984	399073	775956	258495	348610
30-0-0-3	193840	224296	617570	107333	206448
30-0-3-1	301907	449678	763409	337981	337968
30-0-3-2	379346	498310	815645	350548	379354
30-0-3-3	690690	778948	1092715	638426	676248
30-2-0-1	645932	604391	909188	505374	550357
30-2-0-2	577598	561250	772461	465438	492380
30-2-0-3	1378475	1399526	1499448	1354369	1368904
30-2-3-1	1088868	1225485	1364732	1052343	1077584
30-2-3-2	802645	924950	1154086	770189	793561
30-2-3-3	691023	760957	909303	697780	672561

Table 2 Beauty contest — ranking of the algorithms.

Algorithm	Average	Min	Max
GA	1.58	1	4
HC	1.91	0	3
RS	0.03	0	1
SA	3.88	2	4
TS	2.58	2	4

not recommended for the solution of over-resourced problems. Here, all the jobs are allocated and the main objective becomes the reduction of travels. A crossover which concentrates its efforts on preserving tours with a large amount of work done might misguide the search. In this case, a direct crossover using a pure random-based procedure might be more suitable. Secondly, in an over-resourced problem, a significant proportion of the engineers are idle, and have empty tours. In a direct representation, when performing crossover, GA may inherit many empty tours and this is of no real value (i.e. each time this happens, GA is losing one opportunity of mixing good tours). Finally, once the inheritance phase is completed, GA tries to bind each unallocated job to any possible engineer. It might be a better idea to allocate jobs first to existing routes (i.e. engineers with at least one job) and then, if any job remains unallocated, consider idle engineers.

Table 3 shows the amount of exploration performed by the five algorithms on the 50-* problems. The GA column shows the number of crossovers and the RS column shows the number of schedules created in 10 minutes. The HC and TS column give the number of neighbourhoods examined; where a neighbourhood is the number of engineers in the problem plus one (for the conflict list). Therefore, the 50-* problems have a larger neighbourhood than the 30-* problems. The SA column gives the number of temperature reductions. There must be 500 successful moves before the temperature is reduced.

Table 3 Beauty contest — amount of exploration.

	GA	HC	RS	SA	TS
50-0-0-1	10140	23200	1400	1586	22540
50-0-0-2	8040	24400	1400	1588	22920
50-0-0-3	9900	25640	1400	1610	23700
50-0-3-1	44190	28040	4200	1842	23940
50-0-3-2	45200	27120	4300	1858	23300
50-0-3-3	44030	26240	4400	1914	23100
50-2-0-1	14120	18720	1500	1758	16000
50-2-0-2	13300	24560	1400	1802	19960
50-2-0-3	14100	20720	1500	1748	17320
50-2-3-1	55640	22520	4500	2038	19640
50-2-3-2	48810	21040	4000	2006	18700
50-2-3-3	56280	22600	4800	2050	20240

Comparisons should be made downwards through the table not across rows.

The five algorithms fall into three categories — blind, neighbourhood and knowledge- intensive techniques. RS is a blind technique, that does not use the history of the search in any way (with the exception that it compares the current solution with the best solution found so far), and which creates and evaluates a new solution from scratch at each iteration. These two operations are extremely expensive in terms of computational effort and, hence, RS can only visit a limited number of points in the search space.

HC, SA and TS are neighbourhood techniques. When generating new solutions these techniques only need to generate a simple move (i.e. the deletion of a job from an engineer's tour and the addition of that same job to the tour of another engineer) and evaluate the effect of that simple move. These two operations are relatively inexpensive and HC, SA and TS can explore a large number of points in the search space (these points might be quite similar to each other).

GA is a knowledge-intensive technique (when compared to the four other techniques) which combines two solutions into a child and evaluates that child. These two operations are orders of magnitude more expensive than the operations performed by HC, SA and TS. Therefore, GA performs many less iterations (i.e. less exploration but more exploitation) than the three neighbourhood techniques. When creating the child, GA does not have to reschedule the entire solution, as parts (of both parents) are inherited without any alteration. Therefore, GA performs more iterations than RS (in this case, more exploration and also more exploitation). In conclusion, SA is the best technique overall (given 10 minutes). In an environment where the algorithm is only given a limited amount of time (i.e. 10 SPARCstation IPC minutes) SA is a robust technique which performs well both on over-resourced and under-resourced

problems. GA works fairly well on under-resourced problems; however, it is outperformed by the neighbourhood search techniques (SA, HC and TS) on overresourced problems. This is mainly due to the fact that GA does not reduce travel as well as SA, HC and TS. Back to contents

4. CPU intensive runs

In this section, the five techniques - GA (working with a larger population of 2000 chromosomes instead of 400), HC, RS, SA (with the number of successful moves at a given temperature set to 12000 moves, instead of 500), and TS - are given significantly more time to explore the search space. Each algorithm is given the equivalent ¹ of four hours CPU time on a SPARCstation IPC on each problem (five times). The main motivation here is to check if the algorithms can benefit from these longer runs. Is the difference between the solutions provided by the short runs and the solutions provided by the longer runs significant? Is the extra computational effort worth it? Table 4 gives the average cost reached by the five algorithms over the 30-* problem. The five algorithms use the extra time to improve their solutions significantly. GA does not dominate the other techniques on underresourced problems and reduces considerably the gap with (HC, SA and TS) on over-resourced problems. However, SA still remains the best algorithm overall; the ranking among the techniques is SA>,GA≥TS>,HC>,RS (where '>' can be read as 'generally achieves a better score than').

Table 4 CPU intensive experiments — average final cost.

	GA	HC	RS	SA	TS
30-0-0-1	418611	744850	950471	389783	433014
30-0-0-2	191802	521610	747099	193632	240439
30-0-0-3	91073	372232	585131	107356	114528
30-0-3-1	255015	597360	720178	314549	273084
30-0-3-2	283818	618848	770597	337931	312673
30-0-3-3	11376	924910	1060289	609596	605968
30-2-0-1	483638	757418	862319	489156	499902
30-2-0-2	485557	649270	740012	449042	459864
30-0-3-3	611376	924910	1060289	609596	6045968
30-2-0-1	483638	757418	862319	489156	499902
30-2-0-2	485557	649270	740012	449042	459864
30-2-0-3	1327704	1520217	1483255	1339915	1345469
30-2-3-1	1016668	1382046	1314405	1041505	1054138
30-2-3-2	725090	1185889	11126888	750395	739479
30-2-3-3	608013	915757	846271	641898	645515

¹ In fact, these CPU intensive experiments were executed on a DEC Alpha 3000/300; each algorithm was given one hour of CPU time and the DEC Alpha was roughly four times faster than SPARCstation IPC.

Table 5 shows the amount of exploration performed by the different algorithms on the 50-* problems. A comparison between Table 5 and Table 3 (which reports the same data for the short runs) underlines the considerable amount of effort behind each intensive run. For instance, on the 50-0-0-1 problem, RS visited 32170 solutions whereas in the short run, it only had time to generate 1400 solutions. GA performed 361508 crossovers (only 10140 for the short run), HC performed 502400 cycles (only 23200 for the short run) and TS did 583000 cycles (only 22540 for the short run).

Table 5 CPU intensive experiments — amount of exploration.

	GA	HC	RS	SA	TS
50-0-0-1	361508	502400	32170	1216	583000
50-0-0-2	351576	531840	32140	1242	612800
50-0-0-3	326192	550880	32500	1278	612800
50-0-3-1	435360	622560	102480	1374	609000
50-0-3-2	442340	599040	104440	1382	622800
50-0-3-3	446072	603680	107210	1334	628400
50-2-0-1	477620	414800	34920	1324	426400
50-2-0-2	483448	540000	33800	1362	496600
50-2-0-3	479948	398560	34690	1306	425400
50-2-3-1	543000	513520	107820	1466	522200
50-2-3-2	546856	453040	96760	1456	489200
50-2-3-3	523348	521280	116650	1426	537800

The situation is more delicate for SA. The algorithm is performing less iterations during these intensive runs. For instance, on the 50-0-0-1 problem, SA performs 1216 iterations whereas in the short run, it managed 1586 iterations. This is due to the fact that the number of successful moves at a given temperature went from 500 for the short runs up to 12 000 for the intensive runs.

In other words, the main difference for SA between short and intensive runs is that the algorithm is given much more time to reach equilibrium at each temperature; the function used to reduce the temperature is identical in both sets of runs. Some theoretical results (see Hajek [15]) suggest that this annealing schedule might not be the best option, i.e. there is no requirement to attain equilibrium at a succession of reducing temperatures, rather the main condition (for a good annealing schedule) is for the cooling to be carried out sufficiently slowly.

In conclusion, there is no overall champion among the five algorithms (which all benefit from the longer runs). Typically, GA takes advantage of the extra computational effort to provide the best solutions on under-resourced problems. Again, SA is a robust technique which performs well both on under-resourced and over-resourced problems.

5. Other experiences

Full reports of the complete studies performed are available elsewhere [3, 4]. There follows a summary of the work which is not reported elsewhere in this paper.

Several methods of post-optimisation were studied to see if the solutions produced by the search techniques could be simply improved upon. The post-optimisation was performed in two stages; the first being optimisation of travel within tours and the second being an attempt to allocate any jobs still not done. A combination of complete branch-and-bound and heuristic search was used to optimise tours, treating them as small TSPs. As there were relatively few jobs per tour it was usually possible to use a complete search, only resorting to the heuristic if the tour was too long to be solved in a reasonable time. One phenomenon noted was that, as the time windows become tighter, the time required for post-optimisation decreased. This is easily explained by there being less valid job permutations as the time windows get tighter.

The study concluded that RS and GA can benefit greatly from this post-optimisation, TS solutions improve moderately, and HC and SA do not benefit significantly; yet, even with this varied improvement, the techniques' overall ranking does not alter.

Two broadly different GA models (steady-state and generational) were compared with two different chromosomal representations (indirect and direct). Overall, the steady-state model is the best option. The direct GAs totally dominate their indirect opponent (over every single problem). In this particular study, the indirect GA (using a partially mapped crossover operator, i.e. PMX [16]) is not directing its search properly and appears to be just another form of random search. A study was then performed to determine the effect of different population sizes on a given GA. This study showed that larger populations provide a higher level of variety in the pool, and therefore help GA to reach better solutions (by delaying premature convergence). However, the limitations of the schedule builder means that GA spends a prohibitive amount of time creating large populations unless duplicates are accepted. This study also suggests that GA might not benefit from a smart schedule builder.

Finally, GA was enhanced such that it incorporated the post-optimisation heuristic [1] during search. In general, this association GA-local heuristic is fruitful. This is especially true for over-resourced problems where the main objective is to reduce travel. As expected, this hybrid algorithm has a different computational behaviour from the original GA; it is performing less iterations, i.e. less exploration but more exploitation. These results suggest that longer runs could be beneficial.

6. Conclusions

This paper has reported a study on stochastic search techniques applied to the vehicle routing problem. This study took thousands of CPU hours and was wide ranging. In particular, this study has examined:

- the cost benefits of post-optimisation,
- the effects of a new neighbourhood move for HC, SA and TS,
- the relative performances of steady state and generational models of GA,
- comparison of direct versus indirect chromosomal representations and knowledge-based crossover,
- the effects of population size and diversity.

Moreover, the results of two comparative studies were reported. In the 10 minute beauty contest, SA dominates the other competitors, and performed well on all problems. GA does fairly well on under- and critically resourced problems; however, it is outclassed by HC, SA and TS on over-resourced problems, mainly because it cannot reduce travel as well as its three opponents. This result suggests that GA could benefit from longer runs. The CPU intensive experiments (4 hours CPU time instead of 10 minutes) show that indeed GA takes advantages of the extra computational effort and generally it outperforms all the other techniques on under-resourced problems. Again, SA is a robust technique which performs on both under- and over-resourced problems.

In conclusion, when time is available, GA should be preferred for the solution of under-resourced VRPs. However, when the algorithm is only given a limited amount of time or when the VRP is over-resourced, then SA should be preferred.

References

- 1 Buchanan J T, Prosser P and Muller C: 'Some novel approaches to resource allocation problems', Centre for Network and Resource Management, University of Strathclyde (October 1993).
- 2 Tsang E P K: 'Scheduling techniques — a comparative study', *BT Technol J*, **13**, No 1, pp 16—28 (January 1995).
- 3 Prosser P, Muller C and Brind C: 'A preliminary study of stochastic techniques applied to vehicle routing problems', Centre for Network and Resource Management, University of Strathclyde (February 1994).
- 4 Brind C, Muller C and Prosser P: 'A study of stochastic search techniques applied to vehicle routing problems — part 2', Centre for Network and Resource Management, University of Strathclyde (June 1994).
- 5 Smith S F, Ow P S and Potvin J Y: 'OPIS: an opportunistic factory scheduling system', in: 'IEA/AIE90: industrial and engineering applications of artificial intelligence and expert systems', Proceedings of the Third International Conference, Charleston, South Carolina, **1**, (July 1990).
- 6 Fox M S and Smith S F: 'ISIS, a knowledge-based system for manufacturing scheduling', *Expert systems*, **1**, Part 1, pp 25—49 (1984).

- 7 Muller C, Magill E H, Prosser P and Smith D G: 'Stochastic and local techniques for resource allocation problems', Thirteenth international conference on Expert Systems, Avignon, France (May 1992).
- 8 Kirkpatrick S, Gelatt Jr C D and Vecchi M P: 'Optimisation by simulated annealing', *Science*, **220**, pp 671—680.
- 9 Glover F: 'Tabu search — part I', *ORSA Journal on Computing*, **1**, No 3, pp 190—206 (Summer 1989).
- 10 Glover G: 'Tabu search — part II', *ORSA Journal on Computing*, **2**, No 1, pp 4—32 (Winter 1990).
- 11 Ribeiro Filho J L, Treleven P C and Alippi C: 'Genetic-algorithm programming environments', *Computer*, IEEE Computer Society, **27**, No 6, pp 28—43 (June 1994).
- 12 Holland J H: 'Adaptation in natural and artificial systems', University of Michigan Press, Ann Arbor, MI (1989).
- 13 Holland J H: 'Genetic algorithms', *Scientific American*, pp 44—50 (July 1992).
- 14 Srinivas M and Patnaik L M: 'Genetic algorithms: a survey', *Computer*, IEEE Computer Society, **27**, No 6, pp 17—26 (June 1994).
- 15 Hajek B: 'Cooling schedules for optimal annealing', *Mathematics of Operations Research*, **13**, pp 311—329 (1988).
- 16 Goldberg D E and Lingle R: 'Alleles, loci, and the travelling salesman problem', in Grefenstette J (Ed): 'Proc of the First Int Conf on Genetic Algorithms and their Applications', Morgan Kaufman (1985).



Craig Brind is a Research Assistant in the Department of Computer Science at the University of Strathclyde, and is registered for a PhD. He graduated from the University in 1993 with a BSc(Hons) in Computer Science. His current research interests include the application of Artificial Intelligence techniques for tackling resource allocation problems. The work detailed in this paper is being undertaken as part of a sponsorship arrangement between BT and the University of Strathclyde.



Claude Muller has been a research assistant at the University of Strathclyde since 1990. He received a Maitrise Sciences et Techniques (MST) in Control and Computer Sciences from the University de Nancy (France) in 1989 and a diplôme d'études supérieures spécialisées (DESS - equivalent to MSc in UK) in Control, Optoelectronics and Computer Science from the Institut Supérieur d'informatique et automatique de Lorraine (France) in 1990. He is now registered for a PhD.



Patrick Prosser has a BSc and a PhD in computer science. He and Peter Burke (now with BT) were the principal investigators of the distributed asynchronous scheduler (DAS). His current research interests are to do with search, hard problems, the design of algorithms, and the application of that knowledge to real world problems. He has developed numerous algorithms for the constraint satisfaction problem, and has recently performed large scale empirical studies of constraint satisfaction problems and their algorithms.